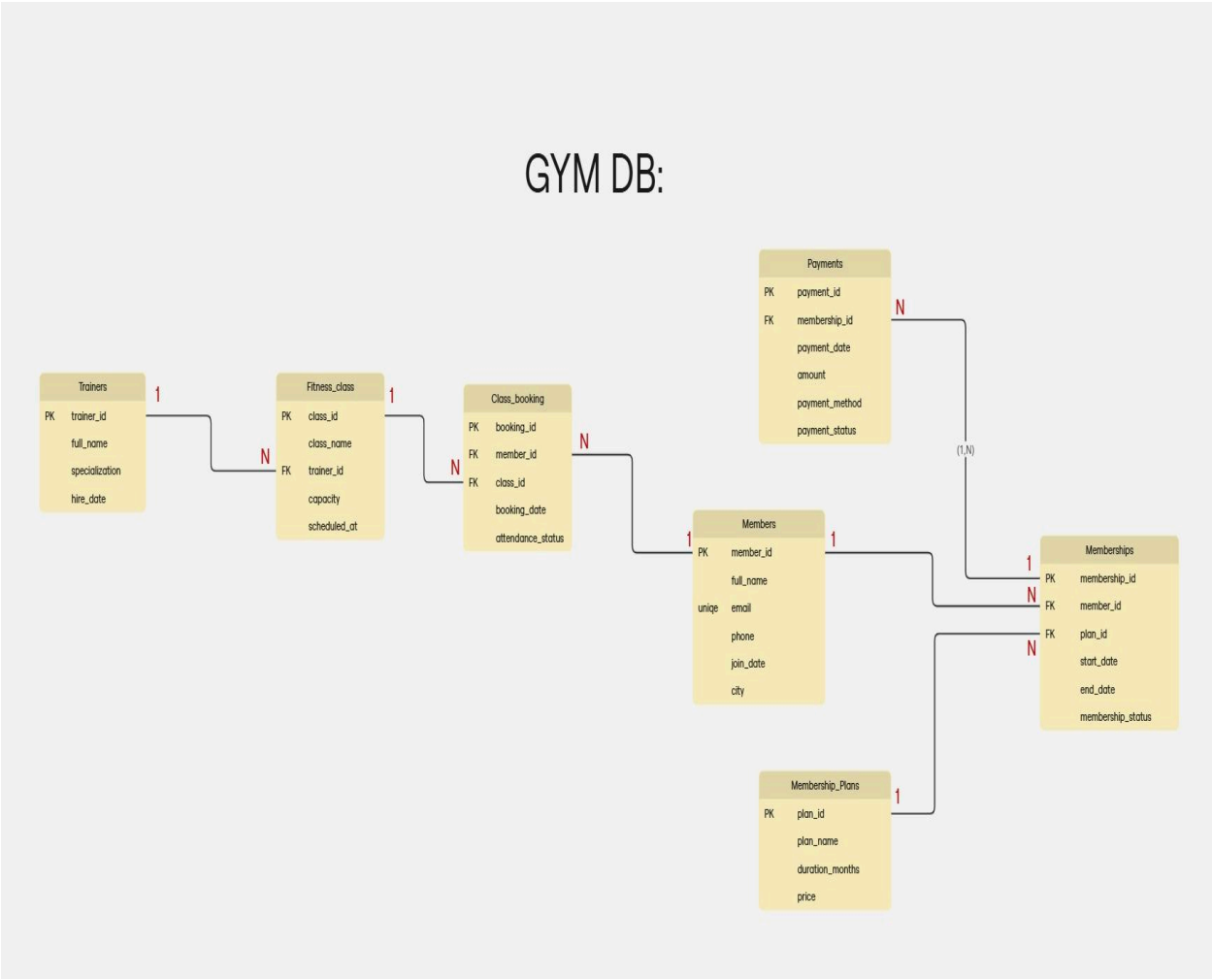
Nada Osama Mohmaed
Assignment : GYM DB

Part A — ERD Design



Part B — Database Implementation

1) Create Tables :

```
-->>>>>>>>>>>>>>>>>>>>>>  1) Create Tables >>>>>>>>>>>>>>>>>>>>>>>>>>>>>
create table members (
    member_id number constraint pk_members primary key,
    full_name varchar2(50) not null,
    email varchar2(100) constraint uq_email_members unique,
    phone varchar2(50),
    join_date date,
    city varchar2(50)
);
create table membership_plan(
    plan_id number constraint pk_membership_plan primary key,
    plan_name varchar2(100) not null,
    duration_months number,
    price number(8,2)
);
create table memberships(
    membership_id number constraint pk_memberships primary key,
    member_id number ,
    plan_id number ,
    start_date date,
    end_date date,
    membership_status varchar2(50),

    constraint fk_memberships_members foreign key (member_id) references
members (member_id),
    constraint fk_memberships_membership_plan foreign key (plan_id) references
membership_plan (plan_id)
);
create table trainers(
    trainer_id number constraint pk_trainers primary key,
    full_name varchar2(100) not null,
    specialization varchar2(100),
    hire_date date
```

```
);
create table fitness_classes(
    class_id number constraint pk_class primary key,
    class_name varchar2(100) not null,
    trainer_id number,
    capacity number,
    scheduled_at date,

    constraint fk_Fclass_trainers foreign key (trainer_id) references trainers
(trainer_id)

);
create table class_booking(
    booking_id number constraint pk_booking primary key,
    member_id number,
    class_id number,
    booking_date date,
    attendance_status varchar2(50),

    constraint fk_Cbooking_members foreign key (member_id) references members
(member_id),
    constraint fk_Cbooking_Fclass foreign key (class_id) references
fitness_classes (class_id)

);
create table payments(
    payment_id number constraint pk_payments primary key,
    membership_id number,
    payment_date date,
    amount number constraint ck_payments_amount check (amount>0),
    payment_method varchar2(50),
    payment_status varchar2(50),

    constraint fk_payments_memberships foreign key (membership_id) references
memberships (membership_id)

);
```

```sql
--Part A – ERD Design
--Part B – Database Implementation
-->>>>>>>>>>>>>>>>>>>>>>>>>>>>>  1) Create Tables >>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>
create table members (
    member_id number constraint pk_members primary key,
    full_name varchar2(50) not null,
    email varchar2(100) constraint uq_email_members unique,
    phone varchar2(50),
    join_date date,
    city varchar2(50)
);
create table membership_plan(
    plan_id number constraint pk_membership_plan primary key,
    plan_name varchar2(100) not null,
    duration_months number,
    price number(8,2)
);
create table memberships(
    membership_id number constraint pk_memberships primary key,
    member_id number ,
    plan_id number ,
    start_date date,
    end_date date,
    membership_status varchar2(50),

    constraint fk_memberships_members foreign key (member_id) references members (member_id),
    constraint fk_memberships_membership_plan foreign key (plan_id) references membership_plan (plan_id)
);
create table trainers(
    trainer_id number constraint pk_trainers primary key,
    full_name varchar2(100) not null,
    specialization varchar2(100),
    hire_date date
);
```

```
);
create table fitness_classes(
    class_id number constraint pk_class primary key,
    class_name varchar2(100) not null,
    trainer_id number,
    capacity number,
    scheduled_at date,

    constraint fk_Fclass_trainers foreign key (trainer_id) references trainers (trainer_id)

);
create table class_booking(
    booking_id number constraint pk_booking primary key,
    member_id number,
    class_id number,
    booking_date date,
    attendance_status varchar2(50),

    constraint fk_Cbooking_members foreign key (member_id) references members (member_id),
    constraint fk_Cbooking_Fclass foreign key (class_id) references fitness_classes (class_id)

);
create table payments(
    payment_id number constraint pk_payments primary key,
    membership_id number,
    payment_date date,
    amount number constraint ck_payments_amount check (amount>0),
    payment_method varchar2(50),
    payment_status varchar2(50),

    constraint fk_payments_memberships foreign key (membership_id) references memberships (membership_id)

);
```

2) Insert Data :

```
-->>>>>>>>>>>>>>>>>>>>>>>>>>>>>  2) Insert Realistic Data >>>>>>>>>>>>>>>>>>>>>>>>>>>
-- ========================================
-- 1. Insert into members
-- ========================================
INSERT INTO members VALUES (1, 'Ahmed Mahmoud', 'ahmed.m@email.com', '01012345678',
TO_DATE('2026-01-10', 'YYYY-MM-DD'), 'Cairo');
INSERT INTO members VALUES (2, 'Fatma Omar', 'fatma.o@email.com', '01198765432',
TO_DATE('2026-02-15', 'YYYY-MM-DD'), 'Alexandria');
INSERT INTO members VALUES (3, 'Youssef Tarek', 'youssef.t@email.com', '01233344455',
TO_DATE('2026-03-01', 'YYYY-MM-DD'), 'Giza');
INSERT INTO members VALUES (4, 'Reem Fouad', 'reem.f@email.com', '01566677788',
TO_DATE('2026-03-10', 'YYYY-MM-DD'), 'Aswan'); -- member with no memberships

-- ========================================
-- 2. Insert into membership_plan
-- ========================================
INSERT INTO membership_plan VALUES (10, 'Basic Monthly Plan', 1, 500.00);
INSERT INTO membership_plan VALUES (20, 'Quarterly Plan', 3, 1350.00);
INSERT INTO membership_plan VALUES (30, 'VIP Annual Plan', 12, 4500.00);
INSERT INTO membership_plan VALUES (40, 'Free Trial Plan', 0, 0.00); -- plan with no
memberships

-- ========================================
-- 3. Insert into trainers
-- ========================================
INSERT INTO trainers VALUES (101, 'Amr Ibrahim', 'Bodybuilding', TO_DATE('2024-05-10',
'YYYY-MM-DD'));
INSERT INTO trainers VALUES (102, 'Sara Ali', 'Yoga & Fitness', TO_DATE('2024-08-15',
'YYYY-MM-DD'));
INSERT INTO trainers VALUES (103, 'Khaled Hassan', 'Cardio & Weight Loss',
TO_DATE('2025-01-20', 'YYYY-MM-DD'));
INSERT INTO trainers VALUES (104, 'Mona Ibrahim', 'Zumba', TO_DATE('2025-06-01',
'YYYY-MM-DD')); -- trainer with no assigned classes
select * from TRAINERS;
delete from TRAINERS;
-- ========================================
```

```sql
-- 4. Insert into fitness_classes
-- ========================================
INSERT INTO fitness_classes VALUES (1, 'Morning Yoga Class', 102, 20, TO_DATE('2026-03-20 08:00:00', 'YYYY-MM-DD HH24:MI:SS'));
INSERT INTO fitness_classes VALUES (2, 'Full Body Cardio', 103, 15, TO_DATE('2026-03-20 18:00:00', 'YYYY-MM-DD HH24:MI:SS'));
INSERT INTO fitness_classes VALUES (3, 'Iron Strength Training', 101, 10, TO_DATE('2026-03-21 19:00:00', 'YYYY-MM-DD HH24:MI:SS'));
INSERT INTO fitness_classes VALUES (4, 'Evening Zumba Class', 102, 25, TO_DATE('2026-03-22 17:00:00', 'YYYY-MM-DD HH24:MI:SS')); -- class with no bookings

-- ========================================
-- 5. Insert into memberships Table
-- ========================================
INSERT INTO memberships VALUES (1, 1, 10, TO_DATE('2026-02-01', 'YYYY-MM-DD'), TO_DATE('2026-03-01', 'YYYY-MM-DD'), 'Active');
INSERT INTO memberships VALUES (2, 2, 20, TO_DATE('2026-02-15', 'YYYY-MM-DD'), TO_DATE('2026-05-15', 'YYYY-MM-DD'), 'Active');
INSERT INTO memberships VALUES (3, 3, 30, TO_DATE('2026-03-01', 'YYYY-MM-DD'), TO_DATE('2027-03-01', 'YYYY-MM-DD'), 'Active');
INSERT INTO memberships VALUES (4, 1, 20, TO_DATE('2026-03-10', 'YYYY-MM-DD'), TO_DATE('2026-06-10', 'YYYY-MM-DD'), 'Pending'); -- membership with no payments

-- ========================================
-- 6. Insert into class_booking Table
-- ========================================
INSERT INTO class_booking VALUES (1, 1, 1, TO_DATE('2026-03-18', 'YYYY-MM-DD'), 'Attended');
INSERT INTO class_booking VALUES (2, 2, 2, TO_DATE('2026-03-18', 'YYYY-MM-DD'), 'Confirmed');
INSERT INTO class_booking VALUES (3, 3, 3, TO_DATE('2026-03-19', 'YYYY-MM-DD'), 'Confirmed');

-- ========================================
-- 7. Insert into payments
-- ========================================
INSERT INTO payments VALUES (1, 1, TO_DATE('2026-02-01', 'YYYY-MM-DD'), 500.00, 'Credit Card', 'Completed');
INSERT INTO payments VALUES (2, 2, TO_DATE('2026-02-15', 'YYYY-MM-DD'), 1350.00, 'Cash',
```

```
'Completed');
INSERT INTO payments VALUES (3, 3, TO_DATE('2026-03-01', 'YYYY-MM-DD'), 4500.00, 'Bank
Transfer', 'Pending');
INSERT INTO payments VALUES (4, 1, TO_DATE('2026-02-05', 'YYYY-MM-DD'), 150.00, 'Credit
Card', 'Failed');

-- Commit changes to database
COMMIT;
```

```sql
-->>>>>>>>>>>>>>>>>>>>>>>>>>>>>  2) Insert Realistic Data >>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>
-- ========================================
-- 1. Insert into members
-- ========================================
INSERT INTO members VALUES (1, 'Ahmed Mahmoud', 'ahmed.m@email.com', '01012345678', TO_DATE('2026-01-10', 'YYYY-MM-DD'), 'Cairo');
INSERT INTO members VALUES (2, 'Fatma Omar', 'fatma.o@email.com', '01198765432', TO_DATE('2026-02-15', 'YYYY-MM-DD'), 'Alexandria');
INSERT INTO members VALUES (3, 'Youssef Tarek', 'youssef.t@email.com', '01233344455', TO_DATE('2026-03-01', 'YYYY-MM-DD'), 'Giza');
INSERT INTO members VALUES (4, 'Reem Fouad', 'reem.f@email.com', '01566677788', TO_DATE('2026-03-10', 'YYYY-MM-DD'), 'Aswan'); -- member

-- ========================================
-- 2. Insert into membership_plan
-- ========================================
INSERT INTO membership_plan VALUES (10, 'Basic Monthly Plan', 1, 500.00);
INSERT INTO membership_plan VALUES (20, 'Quarterly Plan', 3, 1350.00);
INSERT INTO membership_plan VALUES (30, 'VIP Annual Plan', 12, 4500.00);
INSERT INTO membership_plan VALUES (40, 'Free Trial Plan', 0, 0.00); -- plan with no memberships

-- ========================================
-- 3. Insert into trainers
-- ========================================
INSERT INTO trainers VALUES (101, 'Amr Ibrahim', 'Bodybuilding', TO_DATE('2024-05-10', 'YYYY-MM-DD'));
INSERT INTO trainers VALUES (102, 'Sara Ali', 'Yoga & Fitness', TO_DATE('2024-08-15', 'YYYY-MM-DD'));
INSERT INTO trainers VALUES (103, 'Khaled Hassan', 'Cardio & Weight Loss', TO_DATE('2025-01-20', 'YYYY-MM-DD'));
INSERT INTO trainers VALUES (104, 'Mona Ibrahim', 'Zumba', TO_DATE('2025-06-01', 'YYYY-MM-DD')); -- trainer with no assigned classes
select * from TRAINERS;
delete from TRAINERS;
-- ========================================
-- 4. Insert into fitness_classes
-- ========================================
INSERT INTO fitness_classes VALUES (1, 'Morning Yoga Class', 102, 20, TO_DATE('2026-03-20 08:00:00', 'YYYY-MM-DD HH24:MI:SS'));
INSERT INTO fitness_classes VALUES (2, 'Full Body Cardio', 103, 15, TO_DATE('2026-03-20 18:00:00', 'YYYY-MM-DD HH24:MI:SS'));
INSERT INTO fitness_classes VALUES (3, 'Iron Strength Training', 101, 10, TO_DATE('2026-03-21 19:00:00', 'YYYY-MM-DD HH24:MI:SS'));
INSERT INTO fitness_classes VALUES (4, 'Evening Zumba Class', 102, 25, TO_DATE('2026-03-22 17:00:00', 'YYYY-MM-DD HH24:MI:SS')); -- clas


-- ========================================
-- 5. Insert into memberships Table
-- ========================================
INSERT INTO memberships VALUES (1, 1, 10, TO_DATE('2026-02-01', 'YYYY-MM-DD'), TO_DATE('2026-03-01', 'YYYY-MM-DD'), 'Active');
INSERT INTO memberships VALUES (2, 2, 20, TO_DATE('2026-02-15', 'YYYY-MM-DD'), TO_DATE('2026-05-15', 'YYYY-MM-DD'), 'Active');
INSERT INTO memberships VALUES (3, 3, 30, TO_DATE('2026-03-01', 'YYYY-MM-DD'), TO_DATE('2027-03-01', 'YYYY-MM-DD'), 'Active');
INSERT INTO memberships VALUES (4, 1, 20, TO_DATE('2026-03-10', 'YYYY-MM-DD'), TO_DATE('2026-06-10', 'YYYY-MM-DD'), 'Pending'); -- member

-- ========================================
-- 6. Insert into class_booking Table
-- ========================================
INSERT INTO class_booking VALUES (1, 1, 1, TO_DATE('2026-03-18', 'YYYY-MM-DD'), 'Attended');
INSERT INTO class_booking VALUES (2, 2, 2, TO_DATE('2026-03-18', 'YYYY-MM-DD'), 'Confirmed');
INSERT INTO class_booking VALUES (3, 3, 3, TO_DATE('2026-03-19', 'YYYY-MM-DD'), 'Confirmed');

-- ========================================
-- 7. Insert into payments
-- ========================================
INSERT INTO payments VALUES (1, 1, TO_DATE('2026-02-01', 'YYYY-MM-DD'), 500.00, 'Credit Card', 'Completed');
INSERT INTO payments VALUES (2, 2, TO_DATE('2026-02-15', 'YYYY-MM-DD'), 1350.00, 'Cash', 'Completed');
INSERT INTO payments VALUES (3, 3, TO_DATE('2026-03-01', 'YYYY-MM-DD'), 4500.00, 'Bank Transfer', 'Pending');
INSERT INTO payments VALUES (4, 1, TO_DATE('2026-02-05', 'YYYY-MM-DD'), 150.00, 'Credit Card', 'Failed');

-- Commit changes to database
COMMIT;
```

Part C — SQL Queries :

1. Membership details.

```
select ms.membership_id as "Membership ID",m.full_name as "Member Name",mp.plan_name as
"Plan Name", ms.start_date as "Start Date" ,
 ms.membership_status as"Membership Status"
 from MEMBERSHIPS ms inner join members m on ms.MEMBER_ID = m.MEMBER_ID
 join MEMBERSHIP_PLAN mp on ms.PLAN_ID = mp.PLAN_ID;
```

```
146  --================================================================================
147  /*2. Members and memberships. List every member and any membership they have. Members who have never had a
148  membership must still appear. Use LEFT JOIN.*/
149
150  select m.member_id ,ms.membership_id , m.full_name, ms.start_date , ms.end_date , ms.membership_status
151  from members m left join memberships ms
152  on m.member_id = ms.member_id
153  order by m.member_id;
154
155
```

Problems Output Debug Console Terminal Ports Query Result Script Output SQL History Task Monitor

All rows fetched: 5 in 0.048 seconds

	MEMBER_ID	MEMBERSHIP_ID	FULL_NAME	START_DATE	END_DATE	MEMBERSHIP_STATUS
1	1	1	Ahmed Mahmoud	01/02/26	01/03/26	Active
2	1	4	Ahmed Mahmoud	10/03/26	10/06/26	Pending
3	2	2	Fatma Omar	15/02/26	15/05/26	Active
4	3	3	Youssef Tarek	01/03/26	01/03/27	Active
5	4	(null)	Reem Fouad	(null)	(null)	(null)

2. Members and memberships.

```sql
select  m.member_id ,ms.membership_id , m.full_name, ms.start_date , ms.end_date ,
ms.membership_status
from members m left join memberships ms
on m.member_id = ms.member_id
order by m.member_id;
```

```sql
--=========================================================================================
/*2. Members and memberships. List every member and any membership they have. Members who have never had a
membership must still appear. Use LEFT JOIN.*/

select  m.member_id ,ms.membership_id , m.full_name, ms.start_date , ms.end_date , ms.membership_status
from members m left join memberships ms
on m.member_id = ms.member_id
order by m.member_id;
```

Problems | Output | Debug Console | Terminal | Ports | Query Result | Script Output | SQL History | Task Monitor

All rows fetched: 5 in 0.048 seconds

	MEMBER_ID	MEMBERSHIP_ID	FULL_NAME	START_DATE	END_DATE	MEMBERSHIP_STATUS
1	1	1	Ahmed Mahmoud	01/02/26	01/03/26	Active
2	1	4	Ahmed Mahmoud	10/03/26	10/06/26	Pending
3	2	2	Fatma Omar	15/02/26	15/05/26	Active
4	3	3	Youssef Tarek	01/03/26	01/03/27	Active
5	4	(null)	Reem Fouad	(null)	(null)	(null)

3. Classes and bookings.

```
select fc.class_id , fc.class_name,  cb.booking_id , cb.booking_date
from CLASS_BOOKING cb right join FITNESS_CLASSES fc
on cb.CLASS_ID = fc.CLASS_ID;
```

```
156  --================================================================================
157  /*3. Classes and bookings. List every fitness class and any booking made for it. Classes with no bookings must still
158  appear. Write the query using RIGHT JOIN by arranging the tables appropriately.*/
159
160  select fc.class_id , fc.class_name, cb.booking_id , cb.booking_date
161  from CLASS_BOOKING cb right join FITNESS_CLASSES fc
162  on cb.CLASS_ID = fc.CLASS_ID;
163
164  --================================================================================
```

Problems Output Debug Console Terminal Ports Query Result Script Output SQL History Task Monitor

All rows fetched: 4 in 0.066 seconds

	CLASS_ID	CLASS_NAME	BOOKING_ID	BOOKING_DATE
1	1	Morning Yoga Class	1	18/03/26
2	2	Full Body Cardio	2	18/03/26
3	3	Iron Strength Training	3	19/03/26
4	4	Evening Zumba Class	(null)	(null)

4. Trainers and classes.

```sql
select t.trainer_id , full_name,SPECIALIZATION, class_id , class_name
from trainers t left join fitness_classes fc
on t.trainer_id = fc.trainer_id
order by t.trainer_id;
```

```sql
---
--=============================================================================
/*4. Trainers and classes. Display every trainer and the classes they teach, including trainers who have no classes. Use
LEFT JOIN.*/

select t.trainer_id , full_name,SPECIALIZATION, class_id , class_name
from trainers t left join fitness_classes fc
on t.trainer_id = fc.trainer_id
order by t.trainer_id;

--=============================================================================
```

Problems Output Debug Console Terminal Ports **Query Result** Script Output SQL History Task Monitor

All rows fetched: 5 in 0.067 seconds

	TRAINER_ID	FULL_NAME	SPECIALIZATION	CLASS_ID	CLASS_NAME
1	101	Amr Ibrahim	Bodybuilding	3	Iron Strength Training
2	102	Sara Ali	Yoga	1	Morning Yoga Class
3	102	Sara Ali	Yoga	4	Evening Zumba Class
4	103	Khaled Hassan	Cardio Loss	2	Full Body Cardio
5	104	Mona Ibrahim	Zumba	(null)	(null)

5. Plans summary.

```
select mp.plan_name as "Plan Name", count(ms.membership_id) as "Number Of
Memberships", avg(mp.price) as "Average"
from membership_plan mp inner join memberships ms on mp.plan_id = ms.plan_id
group by mp.plan_name , mp.PLAN_ID
order by avg(mp.price) desc;
```

```
173  --================================================================================
174  /*5. Plans summary. For each membership plan that has at least one membership, show the plan name, number of
175  memberships, and average plan price. Sort by average plan price from highest to lowest.*/
176
177  select mp.plan_name as "Plan Name", count(ms.membership_id) as "Number Of Memberships", avg(mp.price) as "Average
178  from membership_plan mp inner join memberships ms on mp.plan_id = ms.plan_id
179  group by mp.plan_name , mp.PLAN_ID
180  order by avg(mp.price) desc;
181
182  --================================================================================
```

roblems Output Debug Console Terminal Ports Query Result Script Output SQL History Task Monitor

All rows fetched: 3 in 0.072 seconds

	Plan Name	Number Of Memberships	Average
1	VIP Annual Plan	1	4500
2	Quarterly Plan	2	1350
3	Basic Monthly Plan	1	500

6. Member booking counts.

```
select m.member_id, m.full_name as "member name", count(cb.class_id) as "total number of
class bookings"
from members m left join class_booking cb
on m.MEMBER_ID = cb.MEMBER_ID
group by m.member_id, m.full_name
order by count(cb.class_id) desc, m.full_name asc;
```

```
182  --================================================================================
183  /*6. Member booking counts. Display every member's name and total number of class bookings, including members with
184  zero bookings. Sort by booking count descending, then member name ascending.*/
185
186  select m.member_id, m.full_name as "member name", count(cb.class_id) as "total number of class bookings"
187  from members m left join class_booking cb
188  on m.MEMBER_ID = cb.MEMBER_ID
189  group by m.member_id, m.full_name
190  order by count(cb.class_id) desc, m.full_name asc;
191
```

Problems Output Debug Console Terminal Ports Query Result Script Output SQL History Task Monitor

All rows fetched: 4 in 0.040 seconds

	MEMBER_ID	member name	total number of class bookings
1	1	Ahmed Mahmoud	1
2	2	Fatma Omar	1
3	3	Youssef Tarek	1
4	4	Reem Fouad	0

7. Successful revenue by plan.

```sql
select plan_name , sum(p.amount) as "total revenue"
from MEMBERSHIP_PLAN mp inner join MEMBERSHIPS ms on mp.PLAN_ID = ms.PLAN_ID
join PAYMENTS p on p.MEMBERSHIP_ID = ms.MEMBERSHIP_ID
where p.payment_status='Completed'
group by plan_name
order by sum(p.amount) desc;
```

```sql
192  --================================================================================
193  /*7. Successful revenue by plan. For each membership plan with successful payments, calculate the total revenue
194  collected. Display the plan name and total revenue, sorted from highest to lowest.*/
195
196  select plan_name , sum(p.amount) as "total revenue"
197  from MEMBERSHIP_PLAN mp inner join MEMBERSHIPS ms on mp.PLAN_ID = ms.PLAN_ID
198  join PAYMENTS p on p.MEMBERSHIP_ID = ms.MEMBERSHIP_ID
199  where p.payment_status='Completed'
200  group by plan_name
201  order by sum(p.amount) desc;
202
```

Problems Output Debug Console Terminal Ports Query Result Script Output SQL History Task Monitor

All rows fetched: 2 in 0.079 seconds

	PLAN_NAME	total revenue
1	Quarterly Plan	1350
2	Basic Monthly Plan	1000

8. Class booking counts.

```sql
select fc.class_id, fc.class_name, count(cb.booking_id) as "total_bookings"
from fitness_classes fc
left join class_booking cb on fc.class_id = cb.class_id
group by fc.class_id, fc.class_name
order by count(cb.booking_id) desc;
```

```sql
--================================================================================
/*8. Class booking counts. Show each fitness class and its number of bookings, including classes with zero bookings. Sort
from most booked to least booked.*/

select fc.class_id, fc.class_name, count(cb.booking_id) as "total_bookings"
from fitness_classes fc
left join class_booking cb on fc.class_id = cb.class_id
group by fc.class_id, fc.class_name
order by count(cb.booking_id) desc;

--================================================================================
```

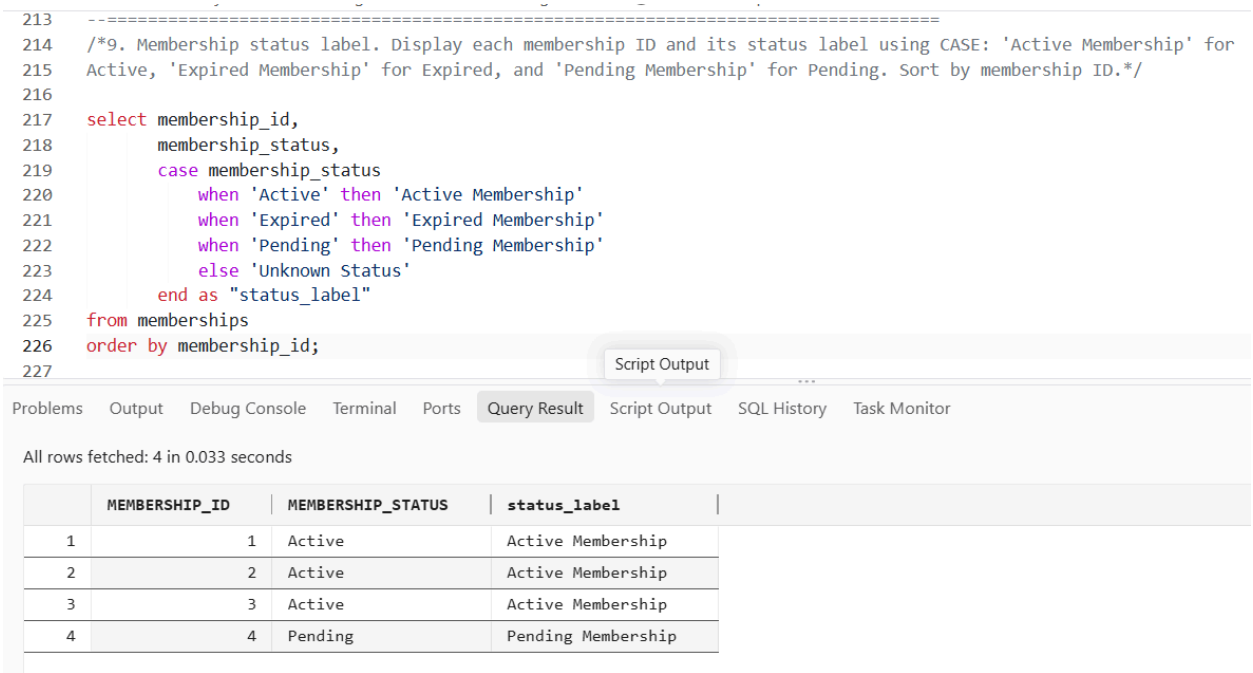
Problems | Output | Debug Console | Terminal | Ports | Query Result | Script Output | SQL History | Task Monitor

All rows fetched: 4 in 0.064 seconds

	CLASS_ID	CLASS_NAME	total_bookings
1	1	Morning Yoga Class	1
2	2	Full Body Cardio	1
3	3	Iron Strength Training	1
4	4	Evening Zumba Class	0

9. Membership status label.

```sql
select membership_id,
       membership_status,
       case membership_status
           when 'Active' then 'Active Membership'
           when 'Expired' then 'Expired Membership'
           when 'Pending' then 'Pending Membership'
           else 'Unknown Status'
       end as "status_label"
from memberships
order by membership_id;
```



```sql
--================================================================================
/*9. Membership status label. Display each membership ID and its status label using CASE: 'Active Membership' for
Active, 'Expired Membership' for Expired, and 'Pending Membership' for Pending. Sort by membership ID.*/

select membership_id,
       membership_status,
       case membership_status
           when 'Active' then 'Active Membership'
           when 'Expired' then 'Expired Membership'
           when 'Pending' then 'Pending Membership'
           else 'Unknown Status'
       end as "status_label"
from memberships
order by membership_id;
```

Script Output

Problems | Output | Debug Console | Terminal | Ports | Query Result | Script Output | SQL History | Task Monitor

All rows fetched: 4 in 0.033 seconds

	MEMBERSHIP_ID	MEMBERSHIP_STATUS	status_label
1	1	Active	Active Membership
2	2	Active	Active Membership
3	3	Active	Active Membership
4	4	Pending	Pending Membership

10. Member contact value.

```sql
INSERT INTO members (member_id,full_name,email,join_date,city) VALUES
                    (5, 'Nada Osama', 'n.o@email.com', TO_DATE('2026-03-10',
'YYYY-MM-DD'), 'cairo'); --for test
--delete from members where full_name ='Nada Osama';

select full_name as "Name", coalesce (phone,email,'No contact details') as contact
from members;
```

```sql
--================================================================================
/*10. Member contact value. Display each member's name and a contact value: phone if available, otherwise email,
otherwise 'No contact details'. Use NVL or an equivalent Oracle-compatible expression.*/

INSERT INTO members (member_id,full_name,email,join_date,city) VALUES
                    (5, 'Nada Osama', 'n.o@email.com', TO_DATE('2026-03-10', 'YYYY-MM-DD'), 'cairo'); --for test
--delete from members where full_name ='Nada Osama';

select full_name as "Name", coalesce (phone,email,'No contact details') as contact from members;

--================================================================================
```

Problems | Output | Debug Console | Terminal | Ports | **Query Result** | Script Output | SQL History | Task Monitor

All rows fetched: 5 in 0.104 seconds

	Name	CONTACT
1	Ahmed Mahmoud	01012345678
2	Fatma Omar	01198765432
3	Youssef Tarek	01233344455
4	Reem Fouad	01566677788
5	Nada Osama	n.o@email.com

11. Amount paid per membership.

```
select ms.membership_id,
       nvl(sum(p.amount), 0) as "total_amount_paid"
from memberships ms
left join payments p on ms.membership_id = p.membership_id
group by ms.membership_id
order by ms.membership_id;
```

```
238  --================================================================================
239  /*11. Amount paid per membership. Display every membership ID and the total amount paid against it. Memberships with
240  no payment records must show 0. Use NVL to handle a missing aggregate result.*/
241
242  select ms.membership_id,
243         nvl(sum(p.amount), 0) as "total_amount_paid"
244  from memberships ms
245  left join payments p on ms.membership_id = p.membership_id
246  group by ms.membership_id
247  order by ms.membership_id;
248
```

Problems Output Debug Console Terminal Ports Query Result Script Output SQL History Task Monitor

All rows fetched: 4 in 0.039 seconds

	MEMBERSHIP_ID	total_amount_paid
1	1	1150
2	2	1350
3	3	4500
4	4	0

12. Trainer class totals.

```sql
select t.full_name as "Trainer Name",
       count(fc.class_id) as "Total Classes"
from trainers t
left join fitness_classes fc on t.trainer_id = fc.trainer_id
group by t.trainer_id, t.full_name
order by count(fc.class_id) desc;
```

```sql
249  --================================================================================
250  /*12. Trainer class totals. Display every trainer's name and the number of classes assigned to them, including trainers with
251  zero classes. Sort by class count descending.*/
252
253  select t.full_name as "Trainer Name",
254         count(fc.class_id) as "Total Classes"
255  from trainers t
256  left join fitness_classes fc on t.trainer_id = fc.trainer_id
257  group by t.trainer_id, t.full_name
258  order by count(fc.class_id) desc;
259
260  --================================================================================
```

Problems | Output | Debug Console | Terminal | Ports | Query Result | Script Output | SQL History | Task Monitor

All rows fetched: 4 in 0.019 seconds

	Trainer Name	Total Classes
1	Sara Ali	2
2	Khaled Hassan	1
3	Amr Ibrahim	1
4	Mona Ibrahim	0

13. Payment status label.

```sql
select payment_id,
       membership_id,
       payment_method,
       amount,
       payment_date,
       case payment_status
           when 'Completed' then 'Completed'
           when 'Pending' then 'Pending'
           when 'Failed' then 'Not completed'
           else 'Unknown'
       end as "payment_status_label"
from payments
order by payment_date desc;
```

```sql
/*13. Payment status label. Display payment ID, membership ID, payment method, amount, payment date, and a CASE
label: 'Completed' for Successful, 'Pending' for Pending, and 'Not completed' for Failed. Sort by payment date newest
first.*/

select payment_id,
       membership_id,
       payment_method,
       amount,
       payment_date,
       case payment_status
           when 'Completed' then 'Completed'
           when 'Pending' then 'Pending'
           when 'Failed' then 'Not completed'
           else 'Unknown'
       end as "payment_status_label"
from payments
order by payment_date desc;
```

Problems Output Debug Console Terminal Ports Query Result Script Output SQL History Task Monitor

All rows fetched: 5 in 0.025 seconds

	PAYMENT_ID	MEMBERSHIP_ID	PAYMENT_METHOD	AMOUNT	PAYMENT_DATE	payment_status_label
1	5	1	Cash	500	20/03/26	Completed
2	3	3	Bank Transfer	4500	01/03/26	Pending
3	2	2	Cash	1350	15/02/26	Completed
4	4	1	Credit Card	150	05/02/26	Not completed
5	1	1	Credit Card	500	01/02/26	Completed

14. Payment statistics.

```sql
select payment_method as "Payment Method",
       sum(amount) as "Total Amount",
       avg(amount) as "Average Amount",
       min(amount) as "Minimum Amount",
       max(amount) as "Maximum Amount",
       count(payment_id) as "Number of Payments"
from payments
group by payment_method;
```

```sql
--================================================================================
/*14. Payment statistics. For each payment method, show the total amount, average amount, minimum amount, maximum
amount, and number of payments.*/

select payment_method as "Payment Method",
       sum(amount) as "Total Amount",
       avg(amount) as "Average Amount",
       min(amount) as "Minimum Amount",
       max(amount) as "Maximum Amount",
       count(payment_id) as "Number of Payments"
from payments
group by payment_method;

--=== Debug Console (Ctrl+Shift+Y) ============================================================
```

Problems Output Debug Console Terminal Ports Query Result Script Output SQL History Task Monitor

All rows fetched: 3 in 0.077 seconds

	Payment Method	Total Amount	Average Amount	Minimum Amount	Maximum Amount	Number of Payments
1	Credit Card	650	325	150	500	2
2	Cash	1850	925	500	1350	2
3	Bank Transfer	4500	4500	4500	4500	1

15. Members with multiple successful payments.

```sql
insert into payments (payment_id, membership_id, amount, payment_date,
payment_method, payment_status)
values (10, 1, 500.00, to_date('2026-03-20', 'YYYY-MM-DD'), 'Cash', 'Completed');

--delete from payments where payment_id=10;

select m.full_name as "Member Name",
       count(p.payment_id) as "Successful Payments Count",
       sum(p.amount) as "Total Successful Amount"
from members m
join memberships ms on m.member_id = ms.member_id
join payments p on ms.membership_id = p.membership_id
where p.payment_status = 'Completed'
group by m.member_id, m.full_name
having count(p.payment_id) >= 2
order by sum(p.amount) desc;
```

```sql
--==============================================================================
/*15. Members with multiple successful payments. Display each member's name, number of successful payments, and
total amount paid from successful payments. Include only members with at least two successful payments, and sort by
total amount paid descending.*/

insert into payments (payment_id, membership_id, amount, payment_date, payment_method, payment_status)
values (10, 1, 500.00, to_date('2026-03-20', 'YYYY-MM-DD'), 'Cash', 'Completed');

select m.full_name as "Member Name",
       count(p.payment_id) as "Successful Payments Count",
       sum(p.amount) as "Total Successful Amount"
from members m
join memberships ms on m.member_id = ms.member_id
join payments p on ms.membership_id = p.membership_id
where p.payment_status = 'Completed'
group by m.member_id, m.full_name
having count(p.payment_id) >= 2
order by sum(p.amount) desc;
```

Problems | Output | Debug Console | Terminal | Ports | Query Result | Script Output | SQL History | Task Monitor

All rows fetched: 1 in 0.038 seconds

	Member Name	Successful Payments Count	Total Successful Amount
1	Ahmed Mahmoud	3	1500